

Maven, Git, Project Structure & Engineering Standards

Complete beginner-friendly notes. Learn how your Spring Boot project is built, managed, tested, versioned, and kept clean.

TODAY'S GOAL

By the end of Day 3, you should understand what Maven does, how dependencies are managed, how a Java project is organized, and how Git is used to safely develop and review changes.

BUILD Maven project	MANAGE Dependencies	ORGANIZE Packages
VERSION Git branches	TEST Build lifecycle	STANDARDIZE Quality rules

Assumption: you know basic Java and have started the Book Catalog API.

Maven is a tool that helps a Java project **build, test, package, and manage libraries**.

Instead of manually downloading JAR files, compiling everything, running tests, and creating a final application, Maven gives you a standard way to do these jobs.

```
Your code
↓
Maven reads pom.xml
↓
Downloads required libraries
↓
Compiles Java
↓
Runs tests
↓
Packages the application
```

Think of Maven as a project assistant

You tell Maven what your project needs in `pom.xml`. Maven takes care of getting those things and running the standard build steps.

`pom.xml` is the main Maven configuration file. It describes your project and the libraries/plugins it uses.

```
<project>
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>book-api</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <dependencies>
    ...
  </dependencies>
</project>
```

You will spend a lot of time understanding the dependencies section, because that is where you tell Maven which libraries your application needs.

03 What is a dependency?

A dependency is simply **code written by someone else that your project needs to use**.

For example, your Spring Boot application needs Spring Web to create REST APIs. Instead of writing all of that code yourself, you add the library as a dependency.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

04 What are Maven coordinates?

Part	Simple meaning
groupId	Who/which organization does this library belong to?
artifactId	What is the library/project called?
version	Which version do you want?
<pre>groupId = org.springframework.boot artifactId = spring-boot-starter-web version = managed by Spring Boot</pre>	

05 Why versions matter

Libraries change over time. A newer version may add features, fix bugs, or change behavior. Your project should use known versions so that the build is predictable.

Reproducible build

If two developers clone the same project, they should be able to run the same build and get the same expected result. Locking down versions is part of making builds predictable.

You do not need to memorize every Maven command today. Focus on the commands you will actually use.

Command	What it does
mvn clean	Removes old build output so you can start fresh.
mvn test	Compiles the project and runs tests.
mvn package	Builds the project and creates the packaged application/JAR.
mvn clean package	Starts clean, then builds and packages.
mvn install	Builds and places the package into your local Maven repository.

Most useful during development

`mvn clean test` is a good habit when you want to make sure the project builds cleanly and tests pass.

When Maven builds your project, it creates a `target/` directory containing compiled classes, test results, and packaged output.

```
book-api/  
■■■ src/  
■■■ pom.xml  
■■■ target/      ← generated by Maven
```

You normally **do not commit target/ to Git**. It is generated output, not source code.

A dependency gives your application code to use. A **plugin gives Maven a job to perform**.

Examples include compiling Java, running tests, creating a JAR, or checking code quality.

Maven

■■■ Dependencies → libraries your code uses

■■■ Plugins → jobs Maven performs

Scope	Simple meaning
compile	Available to normal application code; the usual default.
test	Needed only when compiling/running tests.
provided	Available while compiling, but expected to be supplied by the runtime/environment.

For Day 3, understand why scopes exist. You do not need to memorize every Maven scope.

BOM means **Bill of Materials**. It is a set of compatible library versions managed together.

Spring Boot uses dependency management so that you usually do not have to write a version for every Spring-related dependency.

Why this helps

Without dependency management, you could accidentally choose incompatible versions of libraries. A BOM/dependency management setup keeps related versions aligned.

```

book-api/
  pom.xml
  README.md
  .gitignore
  src/
    main/
      java/
        com/example/bookapi/
      resources/
        application.yml
    test/
      java/
        com/example/bookapi/

```

Location	Purpose
src/main/java	Your Java application code.
src/main/resources	Configuration and other runtime resources.
src/test/java	Automated tests.
pom.xml	Maven project configuration and dependencies.
README.md	Instructions and project documentation.
.gitignore	Tells Git what not to track.

Packages help you group related classes. For the Book Catalog API, keep responsibilities separated:

```

com.example.bookapi
  controller
  service
  dto
  repository
  entity
  exception

```

You do not need every package on Day 3. The important idea is that a class should live near other classes with a similar responsibility.

14 Dependency direction — who should call whom?

```
HTTP request
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

The Controller handles HTTP details. The Service handles application/business work. The Repository handles data access.

Avoid spaghetti

Do not let every class call every other class. Clear direction makes the project easier to understand, test, and change.

15 Example

```
BookController
  ↓ calls
BookService
  ↓ calls
BookRepository
```

A Repository should not normally call the Controller. The Controller is the entry point for HTTP requests.

16 Separation of concerns

Layer	Question it answers
Controller	How do I receive/return an HTTP request?
Service	What should the application do?
Repository	How do I get/save data?
DTO	What data crosses the API boundary?

Git is a version-control tool. It records changes to your code so you can see what changed, go back when needed, and work safely with other developers.

```
graph TD
    A[Your files] --> B[Git tracks changes]
    B --> C[Commits create checkpoints]
    C --> D[Branches let you work separately]
    D --> E[Merge brings completed work together]
```

Think of commits as save points

A commit is a named checkpoint of your project. Good commits make it easier to understand what happened and recover from mistakes.

```
graph TD
    A[Working directory] -- "git add" --> B[Staging area]
    B -- "git commit" --> C[Git history]
```

Working directory: files you are currently changing.

Staging area: the changes you have selected for the next commit.

Commit: a saved checkpoint in Git history.

Command	Plain English
git status	Show what changed.
git add .	Stage current changes.
git commit -m "message"	Create a checkpoint with a message.
git log	See commit history.
git branch	List branches.
git switch -c feature/name	Create and switch to a new branch.
git switch main	Switch to the main branch.
git merge feature/name	Bring a branch's changes into the current branch.
git pull	Get changes from the remote repository and integrate them.
git push	Send local commits to the remote repository.

```
git status
git add .
git commit -m "Add book validation"
git push
```

Before committing, read the diff and make sure the change is intentional.

Why branches?

A branch gives you a separate line of work. Instead of changing the main code directly, you create a feature branch.



This lets you work on a feature without mixing unfinished work into the main branch.

Feature branch workflow

```
git switch main
git pull

git switch -c feature/book-validation

# make changes
git add .
git commit -m "Add book validation"

git push -u origin feature/book-validation
```

Good branch names

Use names that explain the work: `feature/book-search`, `fix/invalid-isbn`, `refactor/book-service`.

Pull request / code review

A pull request is a request to merge your branch into another branch. It gives teammates a place to review the change before it becomes part of the shared code.

A good commit should represent one logical change.

Weak	Better
update stuff	Add validation for book creation
changes	Return 404 when book is missing
final final	Add pagination to book listing

Commit rule

If someone reads only the commit message, they should have a reasonable idea of what changed.

Semantic Versioning commonly looks like MAJOR.MINOR.PATCH.

```
2.4.1
■ ■ ■
■ ■ ■ ■ PATCH → backward-compatible bug fix
■ ■ ■ ■ ■ MINOR → backward-compatible feature
■ ■ ■ ■ ■ ■ MAJOR → breaking/incompatible change
```

You do not need to become a release-management expert today. Just understand that version numbers communicate the kind of change users should expect.

Some files should not be committed: generated build output, IDE settings, local secrets, and machine-specific files.

```
target/
.idea/
*.iml
.env
*.log
```

Never commit passwords, API keys, database credentials, or other secrets.

Formatting does not change what your program does, but it makes code easier to read. A team should not waste code-review time arguing about spaces, braces, or line breaks.

Use one agreed style and let a formatter enforce it.

Static analysis means tools inspect your code without running the application and look for possible problems, bad patterns, or style issues.

Examples include unused imports, suspicious code, complexity, naming problems, and common bugs.

Why automate it?

People miss things. Automated checks catch repeatable problems consistently before code reaches production.

A good project should be buildable by a command, not only by one developer's laptop setup.

```
git clone ...  
mvn clean test  
mvn package
```

If the project can be cloned and built from a clean machine with documented steps, it is much easier for teammates and CI systems to work with.

30 Unit tests vs integration tests

Type	What it tests	Example
Unit test	A small piece of code in isolation.	Does BookService calculate a price correctly?
Integration test	Multiple parts working together.	Does an HTTP request reach the controller and return the expected response?

31 Test package convention

```
src/  
  main/java/com/example/bookapi/  
    controller/  
    service/  
    repository/  
  test/java/com/example/bookapi/  
    controller/  
    service/  
    repository/
```

Keep tests organized similarly to the code they test. This makes them easier to find.

32 Maven test vs package

mvn test	Runs the test phase. Good for quickly checking that tests pass.
mvn package	Runs the build lifecycle up to packaging, including tests by default unless configured otherwise.

33 What should a professional README contain?

Section	Include
Project overview	What the application does.
Requirements	Java/Maven versions and prerequisites.
Setup	How to clone and configure it.
Run	How to start the application.
Test	How to run tests.
API usage	Endpoints and example requests.
Configuration	Environment variables and configuration notes.

34 Example README commands

```
# Run tests
mvn test

# Build the application
mvn clean package

# Run the Spring Boot application
mvn spring-boot:run
```

README test

Imagine a new developer has never seen your project. Could they clone it and understand how to run it without messaging you?

35 Engineering standard: make the easy path the correct path

Good engineering is often about removing unnecessary decisions: standard folder structure, predictable commands, consistent formatting, meaningful commits, automated tests, and clear documentation.

Step	Task
1	Review pom.xml and understand every dependency you currently have.
2	Run mvn clean test and make sure the project builds.
3	Remove generated target/ from Git tracking if it is tracked.
4	Clean up controller/service/dto package organization.
5	Add or improve .gitignore.
6	Separate unit-test and integration-test organization.
7	Create a proper README with setup, run, test, and API usage.
8	Create feature/book-validation branch.
9	Make one logical change and create a meaningful commit.
10	Review your own diff as if you were a senior developer.
11	Merge the completed branch into main.

- Does every dependency in pom.xml have a reason to exist?
- Is generated output excluded from Git?
- Are packages easy to understand?
- Does Controller → Service → Repository direction make sense?
- Are tests located and named consistently?
- Can another developer run the project from the README?
- Are commits small and meaningful?
- Are secrets excluded from the repository?
- Can the project build from a clean checkout?
- Would I be comfortable asking another developer to review this branch?

38	Day 3 interview questions
#	Question
1	What is Maven and why do Java projects use it?
2	What is the purpose of pom.xml?
3	What is a dependency?
4	What is the difference between a dependency and a Maven plugin?
5	What is a transitive dependency?
6	What is dependency scope?
7	What is a BOM?
8	What happens when you run mvn test?
9	What is the target directory?
10	Why should target/ usually be in .gitignore?
11	What is the difference between mvn test and mvn package?
12	Why is package structure important?
13	What is separation of concerns?
14	Why should Controller not contain business logic?
15	What is Git?
16	What is a commit?
17	What is the staging area?
18	Why use branches?
19	What is a pull request?
20	What makes a commit production-friendly?
21	What is a merge conflict?
22	What is semantic versioning?
23	Why are formatting and static analysis useful?
24	Unit test vs integration test?
39	End-of-day self-check

- I can explain Maven without saying only “it builds the project.”
- I can explain what pom.xml, dependencies, plugins, and target/ are.
- I understand the main Maven commands I will use.
- I can explain why package organization matters.
- I understand Controller → Service → Repository direction.
- I can use status, add, commit, branch, switch, merge, pull, and push.
- I can create a feature branch and make a logical commit.
- I understand unit tests vs integration tests.
- I have a useful README.
- I know what should not be committed to Git.
- I can review my own code before opening a pull request.

Day 3 success condition

You should now be able to take your Book Catalog project, build it with Maven, understand its structure, work on it through a Git feature branch, run tests, and explain why your project is organized the way it is.